

Sistemas Operativos



Gestión de Procesos
Multitarea y Planificación



Agenda

- 1** Multitarea
- 2** Estructura del planificador
- 3** Intervención del planificador
- 4** Estrategias de planificación
 - Algoritmos no expropiativos
 - Algoritmos expropiativos
- 5** Colas Multinivel con Realimentación
- 6** Planificador en Linux
 - Earliest Eligible Virtual Deadline First (EEVDF)
- 7** Planificación en Multiprocesadores

Agenda

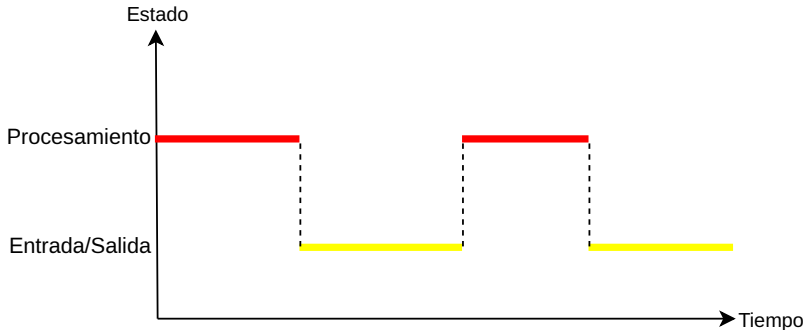


- 1** Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores

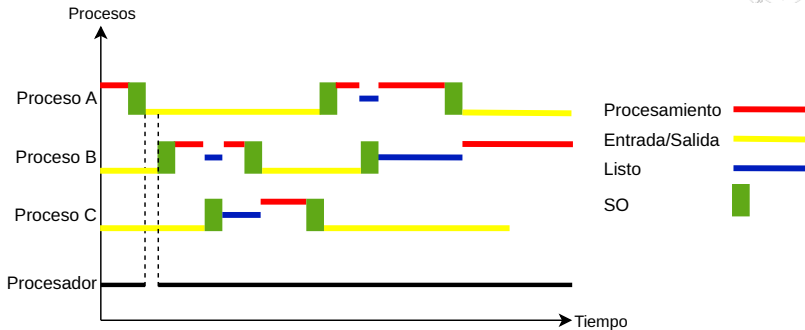


Base de la Multitarea

- Los procesos pasan por fases de E/S y CPU
- Mientras un proceso espera por E/S otro podría utilizar la CPU
- El SO mantiene en memoria varios procesos activos



Ejecución en un sistema multitarea



Proceso nulo o idle

Proceso que mantiene la cpu ejecutando instrucciones cuando no hay ningún otro proceso listo en el sistema. No hace nada útil. Es el menos prioritario.

Ventajas de la multitarea

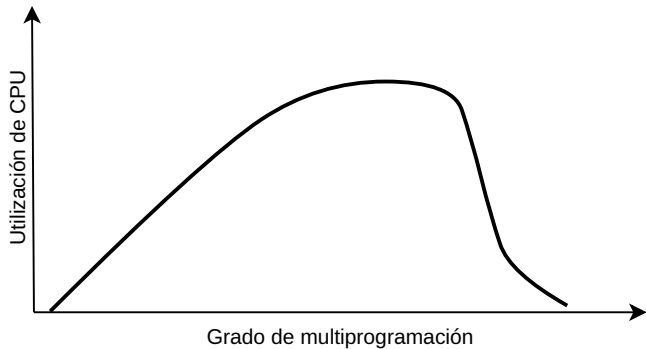


- Facilita la programación, dividiendo los programas en procesos (modularidad)
- Permite el servicio interactivo simultáneo de varios usuarios de forma eficiente
- Aprovecha los tiempos que los procesos pasan esperando a que se completen sus operaciones de E/S
- Aumenta el uso de la CPU



Grado de multiprogramación

- Grado de multiprogramación: n° de procesos activos
- Aumentarlo demasiado puede degradar el rendimiento global por paginación.

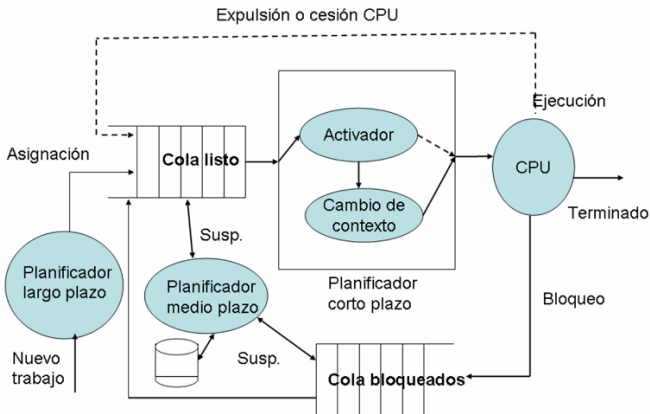


Agenda



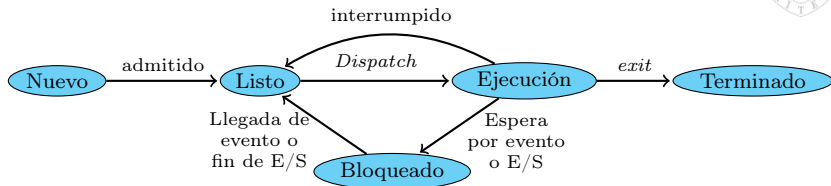
- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores

Niveles de planificación



- **A largo plazo:** añadir procesos a ejecutar (batch)
- **A medio plazo:** añadir procesos a RAM
- **A corto plazo:** qué proceso tiene la CPU

Planificación a corto plazo



Activador o dispatcher: cede el control al siguiente proceso

- Escogido entre los hilos en estado **Listo (ready)**
- Depende de la **política de planificación**
- Cambiar de proceso implica un cambio de contexto

Agenda



- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador**
- 4 Estrategias de planificación
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores

Excepciones/Interrupciones



El SO **siempre** entra a través de una excepción/interrupción

- El tratamiento de la interrupción supone:
 - 1 Detener la ejecución de instrucciones (programa actual)
 - 2 Cambiar a *modo privilegiado*
 - 3 Salvar el estado arquitectónico (en la pila de kernel del proceso)
 - 4 Ejecutar la rutina de tratamiento de excepción/interrupción
- La RTI
 - es código del SO
 - es ejecutada en el contexto del proceso activo
 - finaliza con un proceso de retorno de interrupción que restaura el contexto arquitectónico
- Algunas interrupciones y/o excepciones desencadenan la ejecución del planificador

Activación del Planificador



- 1 Periódicamente, en cada tick del reloj del sistema
 - El HW provee al SO de un contador programable que genera interrupciones periódicas (ticks)
 - Actualiza estadísticas para el algoritmo de planificación
 - Comprueba si le toca planificar otro proceso

Activación del Planificador



- 1 Periódicamente, en cada tick del reloj del sistema
 - El HW provee al SO de un contador programable que genera interrupciones periódicas (ticks)
 - Actualiza estadísticas para el algoritmo de planificación
 - Comprueba si le toca planificar otro proceso
- 2 Por el tratamiento de la interrupción de algún dispositivo de E/S

Activación del Planificador



- 1 Periódicamente, en cada tick del reloj del sistema
 - El HW provee al SO de un contador programable que genera interrupciones periódicas (ticks)
 - Actualiza estadísticas para el algoritmo de planificación
 - Comprueba si le toca planificar otro proceso
- 2 Por el tratamiento de la interrupción de algún dispositivo de E/S
- 3 Por una excepción provocada por el proceso en ejecución:
 - que lo bloquea (ej: fallo de página, llamada al sistema bloqueante)
 - que fuerza su terminación (ej: violación de segmento)
 - que termina voluntariamente el proceso en ejecución (`_exit()`)
 - que cede voluntariamente el procesador (`sched_yield()`)
 - que desbloquea o crea otro proceso (`fork, clone, ...`)

Activación del Planificador



- 1 Periódicamente, en cada tick del reloj del sistema
 - El HW provee al SO de un contador programable que genera interrupciones periódicas (ticks)
 - Actualiza estadísticas para el algoritmo de planificación
 - Comprueba si le toca planificar otro proceso
- 2 Por el tratamiento de la interrupción de algún dispositivo de E/S
- 3 Por una excepción provocada por el proceso en ejecución:
 - que lo bloquea (ej: fallo de página, llamada al sistema bloqueante)
 - que fuerza su terminación (ej: violación de segmento)
 - que termina voluntariamente el proceso en ejecución (`_exit()`)
 - que cede voluntariamente el procesador (`sched_yield()`)
 - que desbloquea o crea otro proceso (`fork, clone, ...`)
- 4 Porque se desbloquea un proceso más *importante* que el actual
 - Sucede realmente por alguno de los mecanismos anteriores

Cambio de Contexto



- Acciones realizadas por el SO para cambiar el proceso en ejecución en una CPU
 - Salvar el contexto del proceso saliente (registros →BCP)
 - Cambiar el estado del proceso saliente (En ejecución →Otro)
 - Configurar la MMU para el espacio de direcciones del proceso entrante:
 - Segmentos o regiones de memoria que puede usar
 - Puntero a la tabla de páginas
 - Flush de TLB
 - Cambiar el estado del proceso entrante, (Listo →En ejecución)
 - Restaurar su contexto (BCP →registros)
 - Realizar el retorno de Interrupción para continuar la ejecución del proceso (en modo usuario)
- Puede llegar a ser una operación bastante costosa



Agenda

- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación**
 - Algoritmos no expropiativos
 - Algoritmos expropiativos
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores

Objetivos

- Optimizar uso de las CPUs
- Minimizar tiempo de espera
- Ofrecer reparto equitativo (justicia)
- Proporcionar grados de urgencia (prioridades)



Métricas



- Parámetros por entidad (proceso o hilo)
 - Tiempo de ejecución (*turnaround*): creación - terminación
 - Tiempo de espera: tiempo total listo y sin CPU
 - Tiempo de respuesta: creación – 1er uso de CPU
- Parámetros globales
 - Porcentaje de utilización del procesador
 - Justicia: equitatividad en el reparto de CPU
 - Productividad: número de trabajos completados por unidad de tiempo



Agenda

- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación**
 - Algoritmos no expropiativos
 - Algoritmos expropiativos
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores

Algoritmos No Expropiativos



El proceso en ejecución conserva la CPU hasta que:

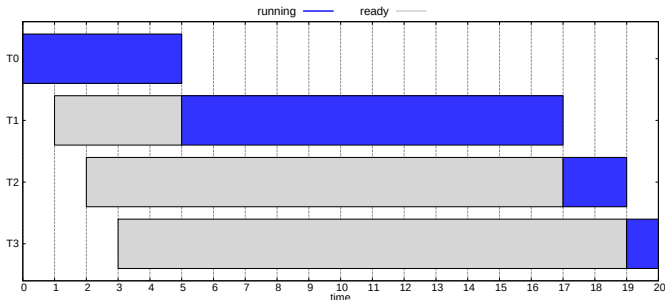
- se bloquea
- cede expresamente la CPU
- termina su ejecución.

First Come First Served (FCFS)



- Se planifican los procesos por orden de entrada en la cola de listos
- Muy sencillo y óptimo en uso de CPU

Proceso o Hilo	Instante de Llegada	Tiempo de CPU (ms)
T0	0	5
T1	1	12
T2	2	2
T3	3	1

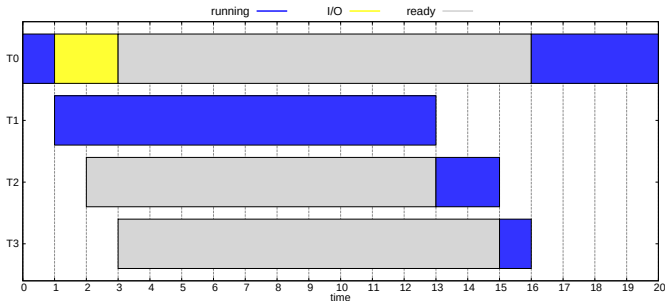




FCFS: tiempos medios altos

- Programas con E/S son encolados al final
- Programas largos afectan al sistema

Proceso o Hilo	Instante de llegada	Tiempo de CPU (ms)
T0	0	5
T1	1	12
T2	2	2
T3	3	1



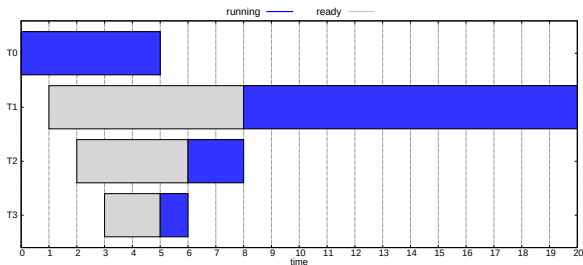


Shortest Job First (SJF)

Se planifica primero el proceso más corto

- Bueno para programas interactivos
- Necesita conocer el perfil de las tareas
- Problemas de inanición

Proceso o Hilo	Instante de llegada	Tiempo de CPU (ms)
T0	0	5
T1	1	12
T2	2	2
T3	3	1

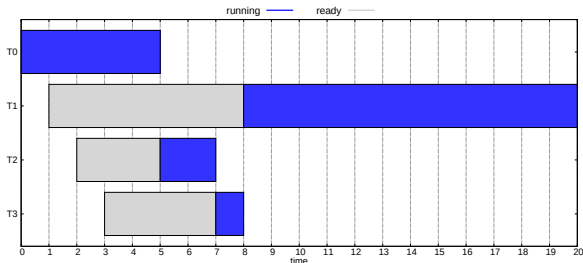




Basado en Prioridades

- Bueno para sistemas con grados de urgencia
- Problema de inanición:
 - Aumento de la prioridad con la edad

Proceso o Hilo	Instante de llegada	Tiempo de CPU (ms)	Prioridad
T0	0	5	4
T1	1	12	3
T2	2	2	1
T3	3	1	2





Agenda

- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación**
 - Algoritmos no expropiativos
 - Algoritmos expropiativos
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores

Algoritmos Expropiativos



- Cuando el planificador lo considera pertinente, cambia el proceso/hilo que hay ejecutando por otro
 - La decisión depende del algoritmo de planificación
- Más adecuados para SO de propósito general
- Manejan bien mezclas de trabajos interactivos y trabajos intensivos en CPU



Round Robin

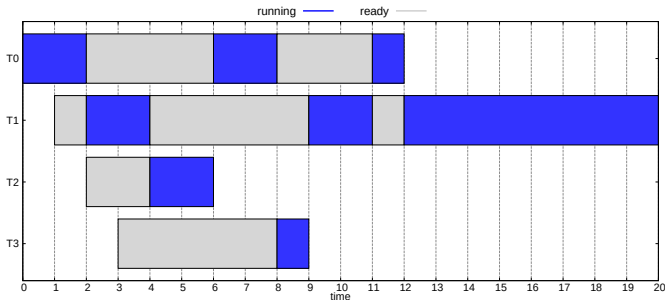
- FCFS + time slice
 - Asignación rotatoria de CPU
 - Se asigna un tiempo máximo de procesador que el proceso puede consumir sin ser expropiado (**time slice** o **cuanto**)
- Uso en sistemas de tiempo compartido
 - Equitativo (mejor por uid que por proceso)

Round Robin: ejemplo (I)



Cuanto: 2

Proceso o Hilo	Instante de llegada	Tiempo de CPU (ms)
T0	0	5
T1	1	12
T2	2	2
T3	3	1

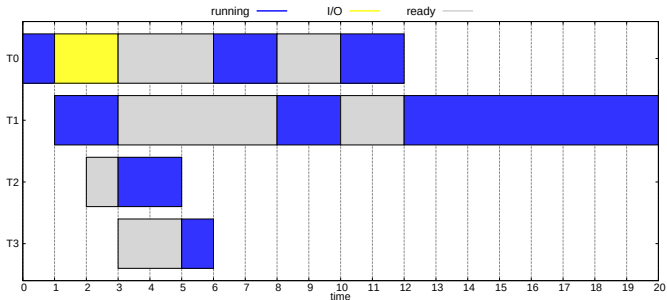


Round Robin: ejemplo (II)



Cuanto: 2

Proceso o Hilo	Instante de llegada	Tiempo de CPU (ms)
T0	0	5
T1	1	12
T2	2	2
T3	3	1

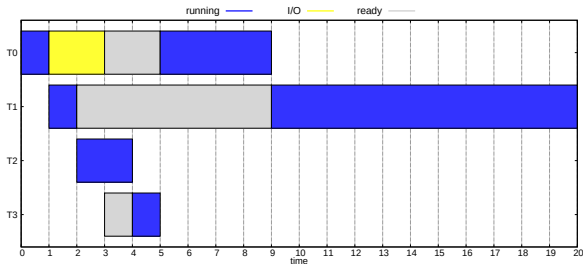


Shortest Remaining Time First (SRTF)



- SJF + time slice variable
 - Bueno para programas interactivos
 - Necesita conocer el perfil de las tareas
 - Problemas de inanición

Proceso o Hilo	Instante de Llegada	Tiempo de CPU (ms)
T0	0	5
T1	1	12
T2	2	2
T3	3	1

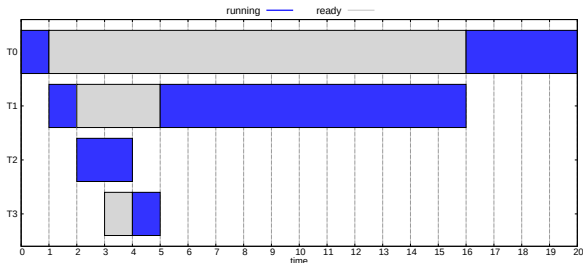




Expropiativo basado en prioridades

- Bueno para sistemas con grados de urgencia
- Problema de inanición:
 - Aumento de la prioridad con la edad

Proceso o Hilo	Instante de llegada	Tiempo de CPU (ms)	Prioridad
T0	0	5	4
T1	1	12	3
T2	2	2	1
T3	3	1	2





Agenda

- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación
- 5 Colas Multinivel con Realimentación**
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores

Motivación



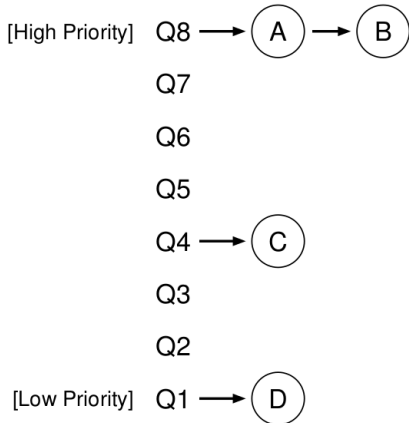
Colas Multinivel con Realimentación (MLFQ)

- Queremos reducir el tiempo de ejecución medio (*turnaround*)
 - Como lo hace SJF
 - Sin conocer a priori el tiempo de ejecución de cada tarea
 - El sistema debe aprender las características de cada tarea
- Queremos reducir el tiempo de respuesta
 - Como lo hace Round Robin
 - Importante para tareas interactivas
- Objetivos contrapuestos



Reglas básicas

- Varias colas con distinto nivel de prioridad
- La prioridad de un proceso es la de la cola a la que está asignado
- Si $\text{Prioridad}(A) > \text{Prioridad}(B)$, A se ejecuta y B no
- Si $\text{Prioridad}(A) == \text{Prioridad}(B)$, se usa RR para planificar la ejecución de A & B
- En el ejemplo:
 - A y B se alternan en uso de la CPU
 - C y D no se ejecutan





Tareas con prioridades dinámicas

- Variar la prioridad de las tareas en función de su comportamiento
 - Si un trabajo repetidamente libera la CPU bloqueándose por E/S debemos asignarle alta prioridad
 - Si una tarea repetidamente usa la CPU de forma intensiva por largos periodos de tiempo, debemos asignarle menor prioridad

El pasado para predecir el futuro

MLFQ intenta *aprender* de los procesos mientras ejecutan, usando así su *historia* para predecir su comportamiento futuro.

Asignación de prioridades: intento 1

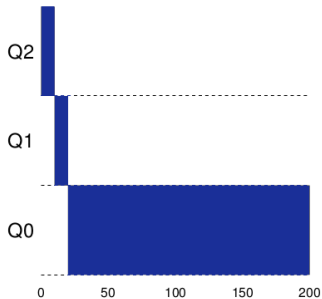


- Cuando una tarea entra en el sistema, se le asigna la máxima prioridad (cola superior)
- Si una tarea consume su cuanto de tiempo completo mientras ejecuta, se *reduce* su prioridad (se mueve a la siguiente cola)
- Si una tarea cede la CPU antes de consumir su cuanto de tiempo, se mantiene en el *mismo* nivel de prioridad.

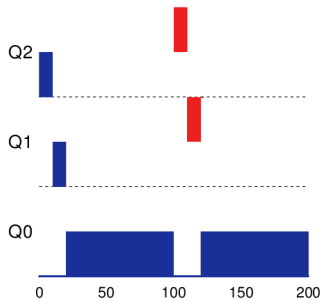
Ejemplo:



Un proceso de larga duración:



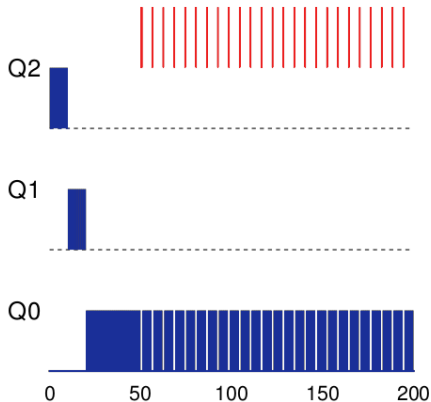
Aparece una tarea más corta:





Ejemplo: tarea interactiva

- Tarea A intensiva en CPU, de larga duración (negro)
- Tarea B interactiva (gris)
 - Ráfagas cortas de CPU, menores que un cuanto, seguidas de fases de E/S





Problemas de la estrategia vista hasta ahora:

- 1 Posible inanición
 - Tareas intensivas en CPU de larga duración
 - Con muchas tareas interactivas, que pueden copar la CPU
- 2 Posibilidad de engañar al planificador
 - Ceder cpu antes de agotar el cuanto
 - Permanecemos en máximo nivel de prioridad
 - Apurando al 99% el cuanto podemos monopolizar la CPU
- 3 No se adapta a los cambios de fase de las tareas
 - Una tarea intensiva en CPU que pasa a una fase interactiva se queda en el nivel menos prioritario

Evitar el problema de inanición



Priority Boost:

- Periódicamente incrementar la prioridad de todos los procesos del sistema
 - Habitualmente, con algún periodo S , todos los procesos se mueven a la cola más prioritaria

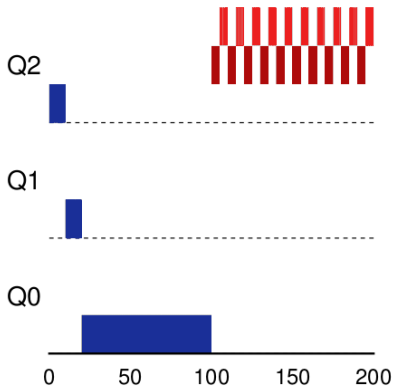
Soluciona dos problemas de golpe:

- Inanición: al pasar periódicamente a las colas más prioritarias todos los procesos podran progresar
- Adaptación a las fases de las tareas. Un cambio a una fase interactiva será detectada y atendida cuando se haga el *Priority Boost*

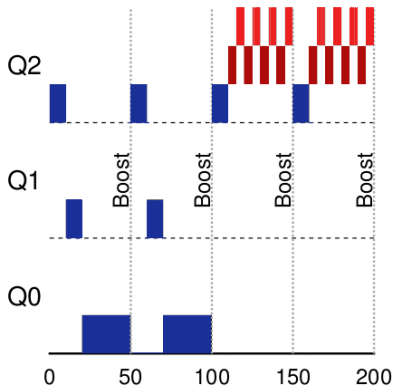


Priority Boost: ejemplo

- Tarea A: larga, intensiva en CPU
- Tareas B y C: interactivas, aparecen a los 100ms



Sin Priority Boost
Inanición de A



Con Priority Boost cada 50ms
A ejecuta cada 50ms

Impedir el engaño al planificador



Es necesario mejorar el *Accounting*:

- Llevar la cuenta del tiempo de CPU total de cada tarea
 - En una o varias ráfagas

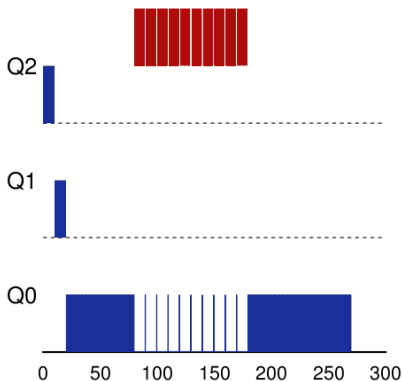
Y modificar las reglas de cambio de prioridades:

- Cuando el tiempo de CPU consumido supere la asignación para ese nivel, la tarea se mueve al siguiente nivel (reduce prioridad)

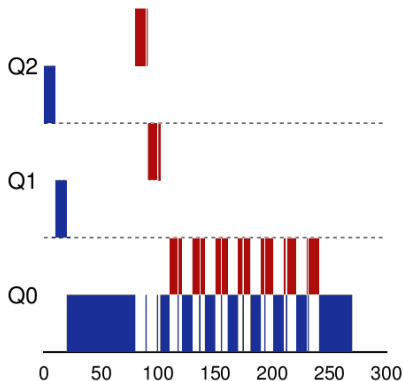


Impedir engaño: ejemplo

- Tarea A: larga, intensiva en CPU
- Tarea B: trata de engañar al planificador, cediendo cpu antes de consumir el cuanto



Sin protección de engaño
B monopoliza la CPU



Con protección de engaño
B no puede monopolizar la CPU

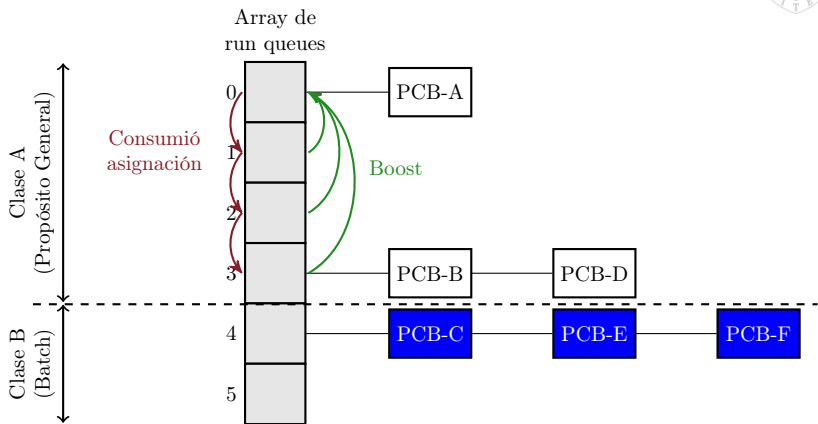
Variaciones habituales



Hay múltiples variantes sobre MLFQ:

- Organización de colas por clases, para distintos tipos de tareas
 - Tareas de kernel, Procesos de usuario, Tareas de tiempo real, Trabajos por lotes (batch)
- Número de colas, cuantos e intervalos de boosting:
 - Solaris: 60 colas para la clase Time Sharing, con cuantos variando de 20ms a 100ms, y Priority Boost en intervalos de 1s
- Mecanismos de cambio prioridad
 - FreeBSD: uso de fórmulas, asignación de prioridad en función del tiempo de CPU y con decaimiento del tiempo de CPU en lugar de Boosting.
 - Solaris: mecanismo de cambio configurable por el administrador del sistema (tablas en ficheros de configuración).
- Admitir consejo de los usuarios para modificar las prioridades
 - Llamada al sistema `nice` en UNIX

MLFQ: estructura ejemplo





Agenda

- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux**
 - Earliest Eligible Virtual Deadline First (EEVDF)
- 7 Planificación en Multiprocesadores

Políticas de planificación



Las tareas en Linux son planificadas de acuerdo a dos parámetros:

- Su política/clase de planificación
- Su prioridad estática

Linux soporta actualmente 6 políticas de planificación:

- `SCHED_DEADLINE` (no POSIX): Earliest Deadline First
- `SCHED_FIFO`: Fifo
- `SCHED_RR`: Round-Robin
- `SCHED_OTHER`/`SCHED_NORMAL`: EEVDF desde la 6.6
- `SCHED_BATCH`: EEVDF desde la 6.6
- `SCHED_IDLE`: EEVDF desde la 6.6

Las clases `SCHED_FIFO`, `SCHED_RR` y `SCHED_DEADLINE` son para tareas de tiempo real

Documentación en `man 7 sched`

Prioridades estáticas



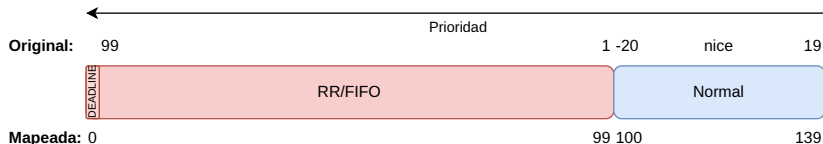
La organización de las clases en prioridades (de mayor a menor) es como sigue:

- Tareas RT: SCHED_DEADLINE > SCHED_RR/SCHED_FIFO
- Tareas de tiempo compartido (no-RT/normal)
 - SCHED_OTHER > SCHED_BATCH > SCHED_IDLE
 - SCHED_OTHER se usa en casi todos los casos.

Dentro de cada clase, Linux planifica tareas por prioridades como sigue:

- SCHED_DEADLINE: Las tareas se seleccionan por deadline.
- SCHED_RR/SCHED_FIFO: Prioridad de 1↓ a 99↑.
- SCHED_OTHER: Prioridad (nice) de -20↑ a 19↓.

Internamente, Linux usa un rango sencillo de 0↑ a 139↓ (0 para SCHED_DEADLINE), que mapea los rangos de prioridades anteriores:





Agenda

- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux**
 - Earliest Eligible Virtual Deadline First (EEVDF)
- 7 Planificación en Multiprocesadores

Earliest Eligible Virtual Deadline First (EEVDF)



- Incorporado en kernel Linux desde v6.8, pero propuesto en 1995
I. Stoica and H. Abdel-Wahab. 1995. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Technical Report. Old Dominion University, USA.

Objetivos EEVDF

- Garantizar una distribución justa del tiempo de CPU considerando la prioridad de los procesos
 - *Proportional-share scheduler*
- Dar soporte a procesos de distinta naturaleza: multimedia (soft real time), interactivos, batch, etc.
- Proporcionar buenos tiempos de respuesta
 - Adecuado para entornos interactivos (GUIs)

Proportional-share scheduler: Idea general



- Si 4 hilos de la misma prioridad estuvieran en el sistema durante 40 ms, cada hilo debería ejecutarse durante 10 ms para asegurar una distribución uniforme (*justicia*)
- *¿Qué hacer si los hilos tuviesen distintas prioridades?*
 - EEVDF asigna a cada proceso un peso, directamente proporcional a la prioridad



Reparto del tiempo de CPU

- Consideremos intervalo arbitrario de ejecución (*sched period*) con n procesos activos $P1, P2, \dots, Pn$
- Reparto de tiempo de CPU en proporcion al peso (w_i) de cada proceso

$$\blacksquare T_{justo}(P) = sched_period_ms \cdot \frac{w_P}{\sum_{i=1}^n w_i}$$

Ejemplo

- 3 procesos A, B y C, con pesos 2,2 y 1, respectivamente
- $sched_period_ms=20ms$
- $T_{justo}(A) = T_{justo}(B) = 8ms$ y $T_{justo}(C) = 4ms$

Proportional CPU sharing con EEVDF



- ¿Cómo reducir la sobrecarga de los cambios de contexto al repartir la CPU?
 - EEVDF usa un cuanto fijo.
 - La selección del siguiente proceso se realiza cada cuanto o cuándo la CPU esté idle
- ¿Cómo repartir la CPU en escenarios muy dinámicos con procesos que se bloquean/activan frecuentemente?
 - EEVDF usa noción de **tiempo virtual**, que transcurre más lento cuantos más procesos hay y más peso tienen
 - $UnidadTiempoVirtual = 1 / \sum_{i=1}^n w_i$
 - Cuando proceso entra o sale del sistema el tiempo virtual se modifica para garantizar justicia

Tiempo de elegibilidad y deadline virtual



- Cada proceso tiene un tiempo de servicio T_S esperado (longitud ráfaga CPU)
 - Si no lo proporciona el usuario, se asume un valor por defecto
- El planificador mantiene un contador de tiempo virtual $V(T)$ por CPU
 - Se incrementa cada vez que un proceso consume un *tick* o una fracción del mismo (se bloquea)

$$V(T) += T / \sum_{i=1}^n w_i$$

EEVDF asigna a cada proceso dos marcas de tiempo virtual

- *ve*: *Tiempo de elegibilidad virtual*
 - Determina cuando el proceso puede ser seleccionado para ejecutar
 - Elegible si $Ve \leq V(T)$
- *vd*: *Deadline virtual*
 - Expresa la urgencia para ejecutar el proceso
 - Función de *ve*, peso (w_P) y $T_{S,P}$

Tiempo de elegibilidad y deadline virtual



- EEVDF escoge para ejecutar, de entre los procesos elegibles (con $ve \leq T_{\text{virtual}}$), aquel con menor deadline virtual (vd)

Actualización de ve y vd

- Cuando proceso entra al sistema
 - $ve = V(T)$
 - $vd = ve + \frac{T_S}{w_P}$
- Cuando un proceso completa T_S
 - El nuevo ve es el antiguo deadline: $ve = vd$
 - Se recalcula un nuevo $vd = ve + \frac{T_S}{w_P}$
- Si un proceso se bloquea antes de completar su T_S , entonces vd se ajusta proporcionalmente
 - $ve = V(T)$ cuando el proceso se desbloquea
 - $vd = ve + \frac{T_{\text{consumido}}}{w_P}$



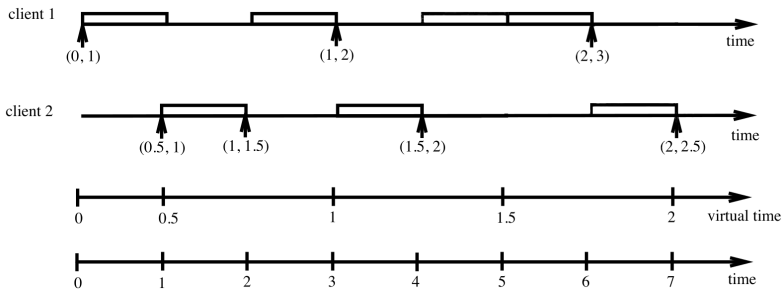
Ejemplo

Proceso o thread	Tiempo de llegada	Peso w_i	Ráfaga T_S (en ticks)
client 1	0	2	2
client 2	1	2	1

- Observar que la tarea 2 llega en el instante 1, y ambas tienen un peso de 2, por tanto:
 - Para $T \in [0, 1)$, cada unidad de tiempo real ejecutado supone un aumento de 0.5 unidades en tiempo virtual.
 - Para $T \geq 1$, cada unidad de tiempo real ejecutado supone un aumento de 0.25 unidades en tiempo virtual.
- En todos los casos:

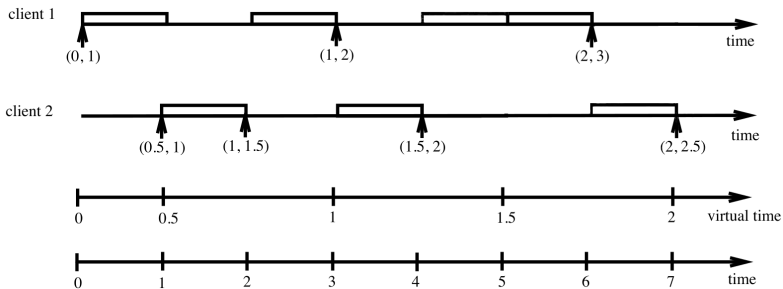
$$(ve, vd) = \left(V(T)_{listo}, V(T)_{listo} + \frac{T_S}{w_P} \right)$$

Ejemplo



- $t = 0, vT = 0$:
 - Llega client 1.
 - $ve(1) = 0, vd(1) = 0 + 2/2 = 1$.
 - client 1 se ejecuta.

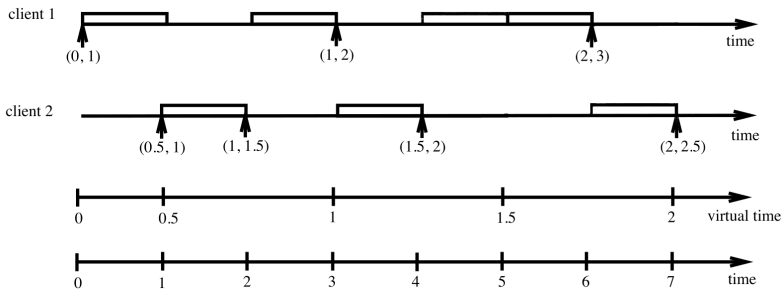
Ejemplo



■ $t = 1, vT = 0.5$:

- client 1 ha ejecutado 1 tick.
- Estado client 1: $ve = 0, vd = 1$ (necesita 1 tick más).
- Llega client 2. $ve(2) = V(T) = 0.5, vd(2) = 0.5 + 1/2 = 1$.
- Ambos elegibles ($ve \leq 0.5$), ambos tienen $vd = 1$. Empate. En el ejemplo se elige ejecutar client 2.

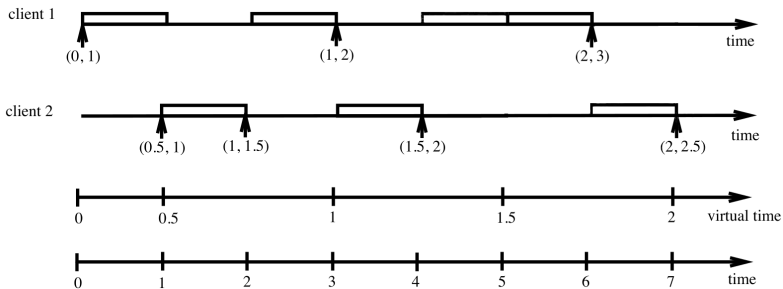
Ejemplo



■ $t = 2, vT = 0.75$:

- client 2 ha ejecutado 1 tick (completa T_S).
- Estado client 1: $ve = 0, vd = 1$ (sigue necesitando 1 tick).
- Estado client 2: $ve(2)_{\text{nuevo}} = vd(2)_{\text{antiguo}} = 1$,
 $vd(2)_{\text{nuevo}} = 1 + 1/2 = 1.5$.
- Decisión: Solo client 1 es elegible ($ve = 0 \leq 0.75$). client 2 no ($ve = 1 > 0.75$). client 1 se ejecuta.

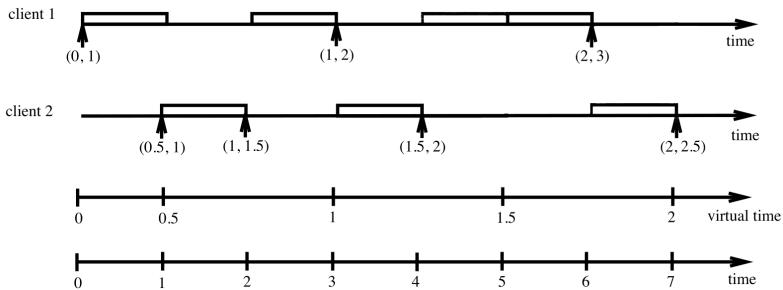
Ejemplo



■ $t = 3, vT = 1$:

- client 1 ha ejecutado 1 tick (completa T_S).
- Estado client 1:
 $ve(1)_{\text{nuevo}} = vd(1)_{\text{antiguo}} = 1, vd(1)_{\text{nuevo}} = 1 + 2/2 = 2$.
- Estado client 2: $ve = 1, vd = 1.5$.
- Decisión: Ambos son elegibles ($ve = 1 \leq V(T) = 1$). $vd(2) = 1.5$ es menor que $vd(1) = 2$. client 2 se ejecuta.

Ejemplo



■ $t = 4, vT = 1.25$:

- client 2 ha ejecutado 1 tick (completa T_S).

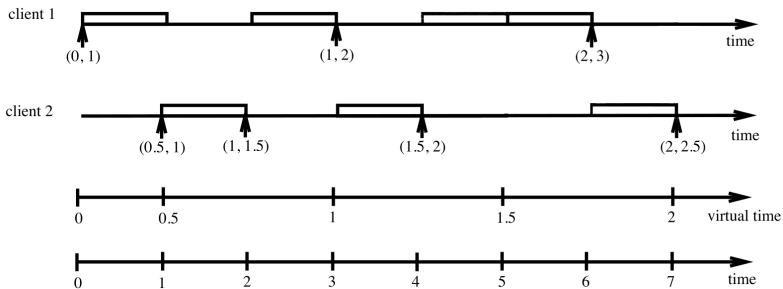
- Estado client 1: $ve = 1, vd = 2$.

- Estado client 2:

$$ve(2)_{\text{nuevo}} = vd(2)_{\text{antiguo}} = 1.5, vd(2)_{\text{nuevo}} = 1.5 + 1/2 = 2.$$

- Decisión: Solo client 1 es elegible ($ve = 1 \leq 1.25$). client 2 no ($ve = 1.5 > 1.25$). client 1 se ejecuta.

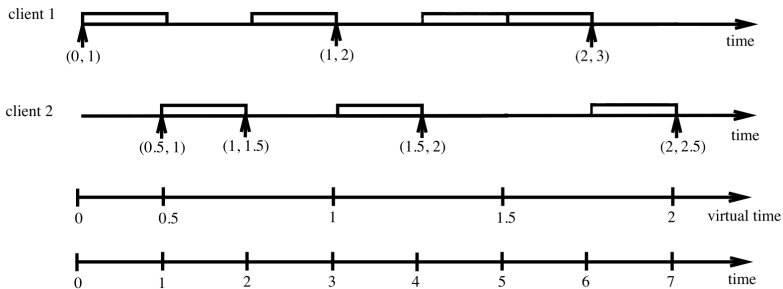
Ejemplo



■ $t = 5, vT = 1.5$:

- client 1 ha ejecutado 1 tick (necesita 1 más).
- Estado client 1: $ve = 1, vd = 2$.
- Estado client 2: $ve = 1.5, vd = 2$.
- Decisión: Ambos elegibles ($ve \leq 1.5$). Ambos tienen $vd = 2$. Empate. client 1 se ejecuta.

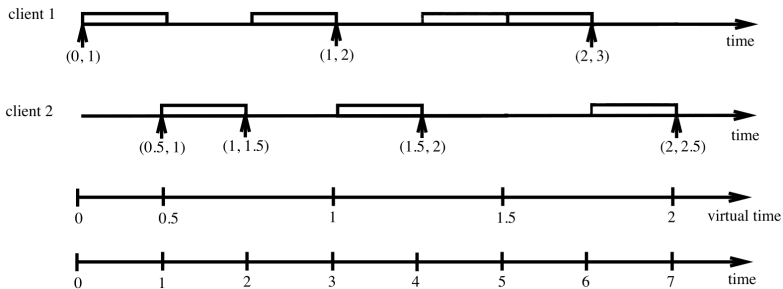
Ejemplo



■ $t = 6, vT = 1.75$:

- client 1 ha ejecutado 1 tick (completa T_S).
- Estado client 1:
 $ve(1)_{\text{nuevo}} = vd(1)_{\text{antiguo}} = 2, vd(1)_{\text{nuevo}} = 2 + 2/2 = 3$.
- Estado client 2: $ve = 1.5, vd = 2$.
- Decisión: Solo client 2 es elegible ($ve = 1.5 \leq 1.75$). client 2 se ejecuta.

Ejemplo



■ $t = 7, vT = 2$:

- client 2 ha ejecutado 1 tick (completa T_S).
- Estado client 1: $ve = 2, vd = 3$.
- Estado client 2:
 $ve(2)_{\text{nuevo}} = vd(2)_{\text{antiguo}} = 2, vd(2)_{\text{nuevo}} = 2 + 1/2 = 2.5$.
- Decisión: Ambos elegibles ($ve = 2 \leq V(T) = 2$). $vd(2) = 2.5$ es menor que $vd(1) = 3$. client 2 se ejecutaría a continuación.

Gestión de escenarios dinámicos



- ¿Qué ocurre si un proceso consume más o menos CPU de lo que le corresponde y se bloquea?
- EEVDF introduce concepto de *lag*
 - Sea t intervalo arbitrario de tiempo transcurrido mientras P activo
 - $\text{Lag}_P(t) = T_{\text{justo},P}(t) - T_{\text{consumido},P}(t)$
 - Si $\text{Lag}_P(t) < 0$ proceso ha consumido más de lo que merece
- El tiempo virtual se ajusta considerando el conjunto de procesos activos (A) y el lag del proceso saliente/entrante:
 - Proceso j sale : $V(T) = V(T) + \text{lag}_j / \sum_{i \in A} w_i$
 - Proceso j entra: $V(T) = V(T) - \text{lag}_j / \sum_{i \in A} w_i$

Gestión de escenarios dinámicos



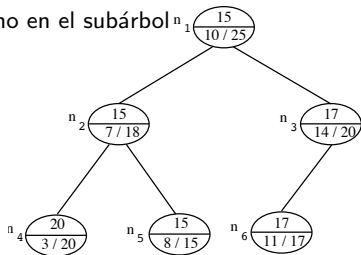
- ¿Qué ocurre si usuario altera la prioridad (peso) de un proceso en ejecución?
 - Equivalente a si saliese un proceso de peso w_i (antiguo) y entrase otro de peso w'_j (nuevo valor)
- Ajuste asociado del tiempo virtual $V(T)$:

$$\blacksquare V(T) = V(T) + \frac{\text{lag}_j}{\left(\sum_{i \in A} w_i\right) - w_j} - \frac{\text{lag}_j}{\left(\sum_{i \in A} w_i\right) - w_j + w'_j}$$



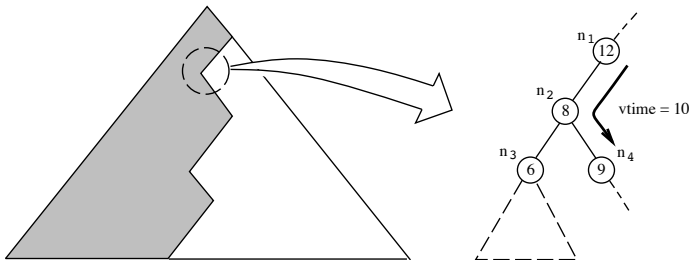
Selección siguiente proceso a ejecutar

- EEVDF mantiene una “lista” de procesos (por cada CPU) ordenada ascendentemente por *ve*
 - Una *run queue* por CPU, formada por los BCPs de procesos asignados a esa CPU (posiblemente con distinta prioridad)
- Por motivos de eficiencia, se usa un *Red-black tree* para implementar la “lista”:
 - Árbol equilibrado: operaciones $O(\log N)$
- Cada nodo almacena *ve* y *vd* del proceso, así como *min_vd*
 - *min_vd*: Deadline virtual mínimo en el subárbol asociado a ese nodo
 - La clave del nodo es *ve*



Selección siguiente proceso a ejecutar

- Para seleccionar el siguiente proceso, EEVDF divide el árbol equilibrado en dos subárboles
 - Nodos elegibles ($ve \leq V(T)$)
 - Nodos no elegibles ($ve > V(T)$)
- Siguiente proceso $\Rightarrow$ Aquel con mínimo vd entre nodos elegibles



EEVDF: conclusiones



Resumen del algoritmo

- El planificador mantiene un contador global (por CPU) de tiempo virtual $V(T)$
 - Transcurre más lentamente que el tiempo real en base al número de procesos activos y su peso
 - Se incrementa a medida que el proceso actual consume tiempo de CPU
- Se asigna a cada proceso un tiempo de elegibilidad (ve) y deadline (vd) virtuales
 - vd denota la urgencia o prioridad dinámica del proceso
 - Deadline más cercano cuanto mayor peso y menor ráfaga de CPU
- Se escoge para ejecutar el proceso elegible ($ve \leq V(T)$) con menor vd
 - Elección eficiente usando un red-black tree
- $V(T)$ se ajusta cuando entran o salen procesos del sistema en base al *lag* del proceso entrante/saliente



Agenda

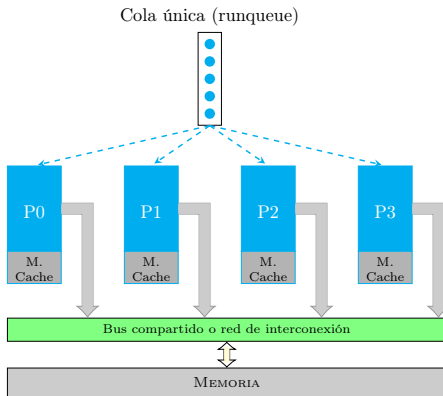
- 1 Multitarea
- 2 Estructura del planificador
- 3 Intervención del planificador
- 4 Estrategias de planificación
- 5 Colas Multinivel con Realimentación
- 6 Planificador en Linux
- 7 Planificación en Multiprocesadores**

Objetivos adicionales



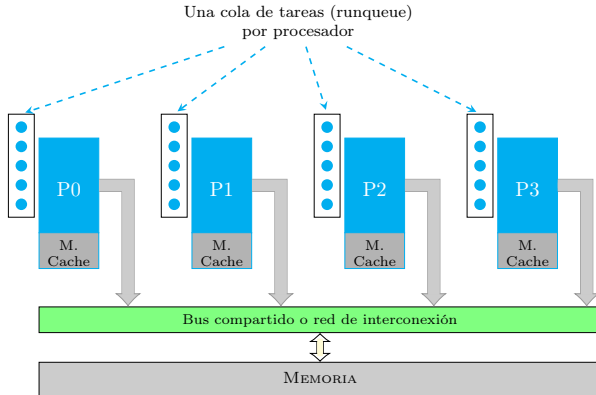
- 1 Garantizar un equilibrio de carga
 - Que no haya un procesador ocioso y otros con mucha carga
- 2 Tener en cuenta la afinidad de procesos y procesadores
 - Importante al replanificar un proceso (evitar migrar)
- 3 Tener en cuenta la compartición de datos entre procesos/hilos si hay varios nodos de memoria (NUMA)
 - Si dos hilos comparten memoria, probablemente sea bueno que compartan todo lo posible la jerarquía de memoria

Opción 1: cola de planificación única



- Bueno para el balanceo de carga
- Malo para afinidad: los procesos pueden ir cambiando de CPU
- La cola es un cuello de botella: problema de escalabilidad

Opción 2: una cola de planificación por CPU



- Mayor escalabilidad
- Equilibrado de carga: periódico o bajo demanda
 - Considera qué procesos pueden/deben migrarse
 - Tiene en cuenta la afinidad
- Utilizada en la mayoría de los sistemas actuales